

Pintos Project 3: Threads

담당 교수 : 김영재

조 / 조원 : 방지혁 20190808

개발 기간 : 2025/10/03 ~ 2025/11/24

I. 개발 목표

기존에는 system call 및 file system을 관여하는 userprog을 구현했었다. 해당 프로젝트에서는 alarm clock, priority scheduling 및 aging, advanced scheduler인 mlfqs scheduler를 구현하고자 한다.

II. 개발 범위 및 내용

A. 개발 범위

1) Alarm Clock

이전의 project 2까지는 thread가 blocked된 state이 경우 busy waiting을 했기에 CPU 자원을 낭비하고 있었다. 이는 매우 비효율적이기에 새롭게 blocked된 thread를 저장하는 list를 만들어 순회하며 thread마다 정해져 있는 tick에 도달하면 깨우게 된다. 이를 통해 앞서 말했듯 CPU 낭비를 막게 된다.

2) Priority Scheduling

이전까지는 Round Robin 형태로 되어 있어, 현재 thread가 시간이 다 되어 작업을 끝내면 list의 마지막 자리로 들어갔다. 뽑을 때는 가장 처음에 있는 thread를 빼냈기에 전혀 priority를 고려하지 않는 구현이었다. 또한, 이 priority가 sema 및 특정 조건에 대해 대기할 때도 고려되지 않았기에 이를 해결합니다. 이를 통해 우선순위를 고려한 scheduling이 구현되며, starvation을 방지하기 위해 priority aging도 구현한다.

3) Advanced Scheduler (추가구현을 한 경우)

MLFQS를 이용한 scheduler를 구현했다. Priority를 기반한 scheduling으로 이 과정에서 nice, load-average, recent-cpu 값을 계산해야한다. 이들은 소수점 연산이 필요하기에 이를 위한 함수들도 구현하고, 해당 값들을 update하고 확인하여 thread를 바꾸는 것도 구현한다. 이를 통해 이전의 경우보다 효율성이 더 높은 scheduling을 구현할 수 있다.

B. 개발 내용

1) Blocked 상태의 스레드를 어떻게 깨울 수 있는지 서술

Blocked 된 thread의 경우, 이들을 관리하는 list가 필요하다. thread_sleep이 수행되었을 때 해당 list에 thread를 추가하고, Timer를 통한 Interrupt가 발생하였을 때 해당 list를 순회하며 thread에 저장되어 있는 시간보다 지났으면 thread_unblock을 통해 해당 thread를 ready list에 넣는다.

2) Ready list에 running thread보다 높은 priority를 가진 thread가 들어올 경우 priority scheduling에 따르면 어떻게 해야 하는지 서술

만약 ready list에 현재 running하고 있는 thread보다 높은 priority를 가진 thread가 들어 오게 될 경우 해당 thread에게 yield해야 한다. 그리고 원래 running하고 있던 thread는 priority에 따라 ready list에 다시 삽입해야 한다. 이를 위해 지속적인 update가 필요하다.

3) Advanced Scheduler에서 priority 계산에 필요한 각 요소를 서술

Priority를 위한 여러 변수에 대한 계산식은 슬라이드에 따랐다. 우선 Priority는 다음과 같은 식으로 계산된다. $PRI_MAX - (recent_cpu / 4) - (nice * 2)$ 이다. Recent_cpu 변수는 thread의 cpu time을 예상하는 변수로 최근에 쓰였을 수록 값이 더 커지는 지표입니다. 가장 처음에 생긴 Thread의 경우 해당 값이 0이지만, 이후의 thread 경우 부모 thread의 값을 상속받는다. 그리고 timer_interrupt가 발생할 때마다 RUNNING 상태 thread의 recent_cpu 값은 1씩 증가한다. 또 상태에 관계없이 매 초마다 $recent_cpu = (2 * load_avg) / (2 * load_avg + 1) * recent_cpu + nice$ 식으로 새롭게 계산된다. 해당 식에서 load_avg는 READY 상태에 있는 thread 개수의 평균을 나타내며, $(59/60) * load_avg + (1/60) * ready_threads$ 식으로 계산된다. 여기서 ready_threads 변수는 READY 상태의 thread에 running 상태의 thread 개수까지 더해 계산한다. 기존의 핀토스는 정수 타입의 계산만 가능했기에 소수점 계산을 위해 새로운 함수들을 구현해야 한다.

III. 추진 일정 및 개발 방법

A. 추진 일정

- 2025/11/03 ~ 2025/11/10: Alarm Clock 관련 부분 구현
- 2025/11/11 ~ 2025/11/18: Priority scheduling 관련 부분 구현
- 2025/11/18 ~ 2025/11/22: MLFQS scheduler 구현 및 debugging
- 2025/11/23 ~ 2025/11/24: 보고서 작성

B. 개발 방법

1) Alarm Clock

기존의 busy waiting 방식에서 벗어나, thread의 상태를 BLOCKED 상태로 변화시키고 일정 시간에 도달하면 READY 상태로 변경되도록 코드를 수정해야 한다. 수정해야 하는 코드 및 헤더로는 threads 디렉토리의 thread.c, thread.h와 devices directory의 timer.c가 있다. 수정해야 하는 자료구조로는 thread 구조체에 int64_t 타입으로 thread가 깨어날 시간을 저장하는 wakeup_time 변수와 sleep한 thread를 저장하는 sleep_thread_list에 들어가기 위한 리스트 요소 struct list_elem sleep_elem를 추가해야 한다. 추가해야 하는 전역

변수로는 앞서 언급했듯 sleep 상태의 thread를 관리하기 위한 sleep_thread_list, 해당 리스트에 있는 thread 중 최소 tick 값을 저장하는 next_wakeup_tick 전역 변수가 있다. 수정 및 추가해야 하는 함수로는 우선 **thread.c**의 **thread_init**이 있다. 해당 함수에서 sleep list 및 next_wakeup_tick을 초기화하는 코드를 추가한다. **Thread_sleep**은 현재 wakeup_time을 인자로 받은 ticks로 설정하고 next_wakeup_time을 갱신하고 thread를 sleep_thread_list에 삽입한다. 마지막으로 thread_block 함수를 호출하여 BLOCKED 상태로 전환시킨다. **Thread_wakeup**의 경우 매 tick마다 호출되기에 list를 순회하며 현재 시간보다 wakeup_time이 더 작거나 같은 thread에 대하여 Thread_unblock()을 호출하여 상태를 READY로 변경시킨다. 그리고 sleep_thread_list에서 해당 스레드를 제거하고 next_wakeup_tick을 다시 계산한다. **Thread_tick**의 경우 매 틱마다 thread_wakeup()을 호출한다. **Timer.c**의 **timer_sleep**은 인터럽트를 비활성화하고, thread_sleep을 호출하고, 다시 인터럽트 상태를 복구한다. **timer_interrupt**는 매 Tick마다 호출되어 ticks를 증가하고 thread_wakeup을 호출하여 깨울 thread를 확인한다. 그리고 앞서 언급한 **thread_tick**을 호출한다.

2) Priority Scheduling

Ready list에 더 높은 우선순위를 가진 thread가 존재할 경우 preemption을 수행시켜야 한다. 우선 수정 및 추가해야 하는 자료구조로는 ready_list로 해당 list가 우선순위를 기준으로 내림차순으로 유지되도록 해야 한다. 이 과정에서 cmp_thread_priority 함수를 새로 만들어 우선순위가 비교되며 들어가도록 한다. Thread_unblock, thread_yield 함수도 list_push_back 대신에 list_insert_ordered를 사용하여 넣는다. Thread_create는 새로운 thread를 생성하고 나서 현재 실행 중인 thread보다 우선 순위가 높으면 thread_yield()를 통한 전환이 이루어질 수 있도록 한다. Thread_set_priority는 기존의 함수 기능에서 우선순위를 변경한 뒤, ready_list에서 가장 높은 우선순위를 가지고 있는 thread보다 우선순위가 낮아지면 마찬가지로 thread_yield()를 호출하여 전환이 이루어질 수 있도록 한다. Sema와 cond의 경우도 waiter 리스트에 우선순위를 고려하여 삽입되도록 한다.

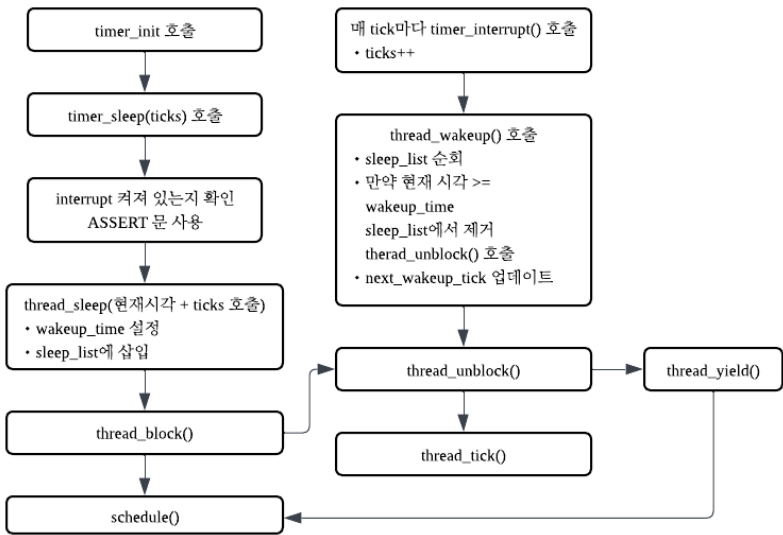
3) Advanced Scheduler

기존처럼 우선순위가 고정되지 않고, 동적으로 설정되도록 구현해야 한다. Thread.h, thread.c를 수정해야 하고, 소수점 연산을 위한 매크로가 있는 fixed_point.h를 추가한다. Thread 구조체에 nice 및 recent_cpu 변수를 추가한다. 그리고, 전역 변수로 시스템 전체의 평균적인 부하를 나타내는 load_avg를 추가해준다. 그리고 priority를 계산하는 mlfs_calc_priority, recent_cpu를 계산하는 mlfs_calc_recent_cpu, load_avg를 계산하는 mlfs_calc_load_avg를 구현한다. 그리고 thread_tick 함수가 thread_mlfqs가 true일 때 매 틱마다 실행 중인 thread에 대하여 recent_cpu 값을 1씩 증가시키고, 매 1초마다 모든 thread의 recent_cpu 및 load_avg가 재계산되도록 한다. 그리고 매 4틱마다 priority를 재계산한다.

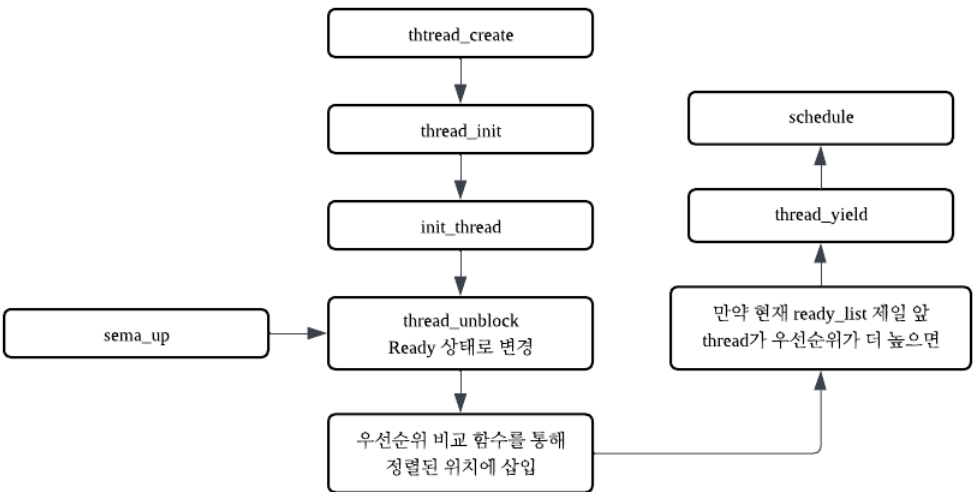
IV. 연구 결과

A. Flow Chart

1) Alarm Clock



2) Priority Scheduling



B. 제작 내용

1) Alarm Clock

timer_sleep

```
void
timer_sleep (int64_t ticks)
{
    ASSERT (intr_get_level () == INTR_ON);
    enum intr_level old_level = intr_disable();

    thread_sleep(timer_ticks() + ticks);
    intr_set_level(old_level);
}
```

기존에는 timer_sleep이 호출되었을 때부터 인자로 받은 시간 동안 계속 확인하며 대기하는 busy waiting 방식이었다. thread_sleep 함수를 호출하는 sleep 및 wakeup 방식으로 변경했다. 그리고 thread_sleep이 끝나고 나서 비활성화한 인터럽트를 다시 활성화하도록 했다.

자료구조 수정 & 추가 및 초기화

```
struct thread {
    /* Owned by thread.c. */
    tid_t tid; /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    char name[16]; /* Name (for debugging purposes). */
    uint8_t* stack; /* Saved stack pointer. */
    int priority; /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem; /* List element. */
    int64_t wakeup_time; /* 깨워야 하는 tick */
    struct list_elem sleep_elem;
    int nice;
    int recent_cpu;
#ifdef USERPROG
    /* sleep queue of threads */
    static struct list sleep_thread_list;
    static int64_t next_wakeup_tick;
#endif
}
```

전역변수로 잠든 thread를 관리하는 리스트인 sleep_thread_list와 자고 있는 thread의 wakeup_time 중 가장 작은 시간을 저장할 next_wakeup_tick을 선언한다. Thread 구조체에 thread가 깨어나야 할 시간을 저장하는 필드인 wakeup_time과 sleep_list에 넣을 때 사용되는 list element인 sleep_elem도 추가해준다. 그리고 언급한 전역 변수들도 thread_init 함수에서 초기화 해준다.

```
void
thread_init (void)
{
    ASSERT (intr_get_level () == INTR_OFF);

    lock_init (&tid_lock);
    list_init (&ready_list);
    list_init (&all_list);
    list_init (&sleep_thread_list);

    /* Set up a thread structure for the running thread. */
    initial_thread = running_thread ();
    init_thread (initial_thread, "main", PRI_DEFAULT);
    initial_thread->status = THREAD_RUNNING;
    initial_thread->tid = allocate_tid ();
    next_wakeup_tick = INT64_MAX;
    initial_thread->recent_cpu = INT_TO_FP(0);
    initial_thread->nice = 0;
    load_avg = INT_TO_FP(0);
}
```

thread_sleep

```
void thread_sleep(int64_t ticks) {
    struct thread *cur = thread_current();
    if (cur == idle_thread) return;

    cur->wakeup_time = ticks;

    if (next_wakeup_tick > ticks) {
        next_wakeup_tick = ticks;
    }

    list_push_back(&sleep_thread_list, &cur->sleep_elem);
    thread_block();
}
```

Idle thread의 경우 재우지 않고, 현재 thread에 대해 wakeup_time 필드에 깨어날 시각을 저장한다. 가장 작은 wakeup_time을 저장하는 next_wakeup_tick 전역 변수에 대해 더 작은 시간이 있다면 업데이트 해준다. list_push_back을 통해 sleep_list에 추가한다. 그리고 thread_block 함수를 호출한다. Thread_block 함수에서는 현재 thread를 BLOCKED 상태로 만들고 schedule()을 호출한다.

Timer_interrupt

```
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
    ticks++;
    /* Wake up sleeping threads */
    thread_wakeup();
    thread_tick ();
}
```

해당 함수는 매 타이머 인터럽트마다 호출되는데 sleep_thread_list를 순회하며 잠든 thread를 깨워줄 thread_wakeup 함수를 추가적으로 구현하여 이를 호출하도록 수정한다.

다음 장에 이어서

Thread_wakeup

```
void thread_wakeup(void) {
    int64_t current_tick = timer_ticks();
    if (next_wakeup_tick > current_tick) return;

    int64_t new_min = INT64_MAX;

    for (struct list_elem *e = list_begin(&sleep_thread_list); e != list_end(&sleep_thread_list);) {
        struct thread *t = list_entry(e, struct thread, sleep_elem);

        if (t->wakeup_time <= current_tick) {
            e = list_remove(e);
            thread_unblock(t);
        }

        else {
            if (new_min > t->wakeup_time) new_min = t->wakeup_time;
            e = list_next(e);
        }
    }
    next_wakeup_tick = new_min;
}
```

현재 시각을 확인하고, 가장 작은 wakeup_time보다 작으면 종료한다. 그렇지 않을 경우 sleep_thread_list를 순회하며 각 thread에 대하여 만약 wakeup_time이 현재 시간보다 작으면 list_remove를 통해 제거하고 thread_unblock으로 깨운다. 현재 시간보다 크다면 최소 wakeup_time을 갱신하고 다음 elem로 이동하며 순회한다. 그리고 마지막으로 전역 변수인 next_wakeup_tick을 새로운 최소 값으로 업데이트해주면 된다.

Thread_unblock

```
void
thread_unblock (struct thread *t)
{
    enum intr_level old_level;

    ASSERT (is_thread (t));

    old_level = intr_disable ();
    ASSERT (t->status == THREAD_BLOCKED);
    list_insert_ordered(&ready_list, &t->elem, cmp_thread_priority, NULL);
    t->status = THREAD_READY;
    intr_set_level (old_level);
}
```

우선 인터럽트를 비활성화하고 Ready_list에 추가적으로 구현한 cmp_thread_priority 함수를 사용하여 우선순위 순으로 삽입하고 Blocked 상태의 thread를 ready 상태로 바꿔준다. 그리고 다시 인터럽트를 복원한다.

Cmp_thread_priority

```
bool cmp_thread_priority(const struct list_elem *a, const struct list_elem *b, void *aux UNUSED) {
    int priority_a = list_entry(a, struct thread, elem)->priority;
    int priority_b = list_entry(b, struct thread, elem)->priority;
    return priority_a > priority_b;
}
```

두 thread의 priority를 비교하는 함수로 a의 우선순위가 더 높으면 true, 그 외의 경우 false를 반환한다. 우선순위가 높은 게 앞쪽으로 정렬되도록 하는 도우미 함수다.

2) Priority Scheduling

Thread_create 수정

```
/* Add to run queue. */
thread_unblock (t);

/* 우선순위 비교 */
if (t->priority > thread_get_priority()) thread_yield();
return tid;
}
```

새 thread를 만들 경우, thread_unblock()으로 ready_list에 우선 순위로 삽입하고 나서 현재 thread와 새 thread의 우선순위를 비교하여 새로운 thread가 더 높을 경우 thread_yield()를 호출하여 CPU를 양보할 수 있도록 합니다.

Thread_yield

```
void
thread_yield (void)
{
    struct thread *cur = thread_current ();
    enum intr_level old_level;

    ASSERT (!intr_context ());

    old_level = intr_disable ();
    if (cur != idle_thread)
        list_insert_ordered(&ready_list, &cur->elem, cmp_thread_priority, NULL);
    cur->status = THREAD_READY;
    schedule ();
    intr_set_level (old_level);
}
```

인터럽트를 비활성화하고 idle thread가 아니라면 ready_list에 우선순위로 삽입되도록 한다. 상태를 ready로 바꾸고, schedule함수를 호출하여 context switch가 이루어지도록 한다. 그 결과 가장 높은 우선순위의 thread가 실행될 수 있다.

Thread_set_priority

```
void
thread_set_priority (int new_priority)
{
    if (thread_mlfqs) return;
    thread_current ()->priority = new_priority;
    // ??
    if (!list_empty(&ready_list)) {
        struct thread *highest_priority_thread = list_entry(list_front(&ready_list), struct thread, elem);
        if (new_priority < highest_priority_thread->priority) thread_yield();
    }
}
```

우선순위를 변경하는 함수로 Mlfqs 스케줄링일 경우, 직접 우선순위 변경이 불가능하기에 종료한다. 그외의 경우 우선순위를 변경하고 ready_list가 비워져 있지 않을 경우 우선순위가 가장 높은 맨 앞의 thread와 비교해서 내 우선순위가 더 낮으면 thread_yield를 호출하여 양보한다.

다음 장에 이어서

Thread_preemption

```
void thread_preemption(struct thread* cur) {
    if (list_empty(&ready_list)) return;
    struct thread* thread_to_check =
        list_entry(list_front(&ready_list), struct thread, elem);
    if (cur->priority < thread_to_check->priority) {
        thread_yield();
    }
}
```

Sema_up 함수에서 ready_list를 직접 참조하기 어려워 만든 함수로 thread_set_priority 함수처럼 ready_list가 비어있으면 return 하고, 우선순위가 가장 높은 맨 앞의 스레드와 비교하여, 해당 thread가 더 높으면 thread_yield를 호출하여 양보한다.

Sema_up

```
void sema_up(struct semaphore* sema) {
    enum intr_level old_level;

    ASSERT(sema != NULL);

    old_level = intr_disable();
    if (!list_empty(&sema->waiters)) {
        list_sort(&sema->waiters, cmp_thread_priority, NULL);
        thread_unblock(
            list_entry(list_pop_front(&sema->waiters), struct thread, elem));
    }
    sema->value++;
    struct thread* cur = thread_current();
    thread_preemption(cur);
    intr_set_level(old_level);
}
```

List_sort로 waiters 리스트를 재정렬하고, 가장 높은 우선순위의 thread를 꺼내서 thread_unblock으로 깨운다. 앞서 서술한 thread_preemption을 호출하여 깨운 thread가 더 높은 우선순위를 가지면 선점할 수 있도록 한다.

Sema_down

```
void sema_down(struct semaphore* sema) {
    enum intr_level old_level;

    ASSERT(sema != NULL);
    ASSERT(!intr_context());

    old_level = intr_disable();
    while (sema->value == 0) {
        list_insert_ordered(&sema->waiters, &thread_current()->elem,
                           cmp_thread_priority, NULL);
        thread_block();
    }
    sema->value--;
    intr_set_level(old_level);
}
```

Waiters 리스트에 우선순위 순으로 삽입되도록 변경했다.

Cond_signal

```
void cond_signal(struct condition* cond, struct lock* lock UNUSED) {
    ASSERT(cond != NULL);
    ASSERT(lock != NULL);
    ASSERT(!intr_context());
    ASSERT(lock_held_by_current_thread(lock));

    if (!list_empty(&cond->waiters)) {
        list_sort(&cond->waiters, cmp_semaphore_priority, NULL);
        sema_up(&list_entry(list_pop_front(&cond->waiters), struct semaphore_elem, elem)->semaphore);
    }
}
```

List_sort로 Waiters 리스트를 우선순위로 재정렬해서 가장 높은 우선순위의 semaphore_elem를 꺼낸다. 그리고 해당 semaphore에 sema_up을 호출하여 스레드를 깨울 수 있다.

Cmp_semaphore_priority

```
bool cmp_semaphore_priority(const struct list_elem* a, const struct list_elem* b, void* aux UNUSED) {
    struct semaphore_elem* sa = list_entry(a, struct semaphore_elem, elem);
    struct semaphore_elem* sb = list_entry(b, struct semaphore_elem, elem);
    struct thread* thread_a =
        list_entry(list_front(&sa->semaphore.waiters), struct thread, elem);
    struct thread* thread_b =
        list_entry(list_front(&sb->semaphore.waiters), struct thread, elem);
    int thread_priority_a = thread_a->priority;
    int thread_priority_b = thread_b->priority;
    return thread_priority_a > thread_priority_b;
}
```

Semaphore 간 우선순위를 비교하는 함수로 각 semaphore_elem에서 waiters 리스트에 접근한다. 그리고, 두 thread간의 우선순위를 비교한다.

Thread_tick

```
void
thread_tick (void)
{
    struct thread *t = thread_current ();

    /* Update statistics. */
    if (t == idle_thread)
        idle_ticks++;
#ifdef USERPROG
    else if (t->pagedir != NULL)
        user_ticks++;
#elseif
    else
        kernel_ticks++;

    /* Enforce preemption. */
    if (++thread_ticks >= TIME_SLICE)
        intr_yield_on_return ();

    // thread_wakeup();
    /* Project #3 */
#ifdef USERPROG
    if (thread_prior_aging == true) {
        thread_aging();
    }
}
```

매 틱마다 호출되는 함수로 thread_prior_aging 플래그가 true이면 thread_aging을 호출하여 Thread가 매 틱마다 우선순위가 올라가도록 한다.

Thread_aging

```
void thread_aging(void) {
    for (struct list_elem* e = list_begin(&ready_list); e != list_end(&ready_list);
         e = list_next(e)) {
        struct thread* t = list_entry(e, struct thread, elem);
        t->priority++;
        if (t->priority > PRI_MAX) t->priority = PRI_MAX;
    }
    list_sort(&ready_list, cmp_thread_priority, NULL);
}
```

Ready_list 내의 모든 thread를 순회하며, 우선 순위를 1씩 증가시켜 starvation을 방지한다. PRI_MAX 초과를 방지하고, list_sort로 재정렬시킨다.

3) Advanced Scheduler (MLFQS)

Fixed_point.h

```
#ifndef THREADS_FIXED_POINT_H
#define THREADS_FIXED_POINT_H

// p is integer & q is fraction
// 32-bit 1 bit => sign | 17 bits => integer(p) | 14 bits => fraction(q)

#define F (1 << 14) // 2^14로 1.0 의미

// int to fp
#define INT_TO_FP(n) ((n) * F)

#define FP_TO_INT_ZERO(x) ((x) / F)
#define FP_TO_INT_NEAREST(x) ((x) >= 0 ? ((x) + F / 2) / F : ((x) - F / 2) / F)
// x와 y 둘 다 fp일 때 더하기
#define FP_ADD_FP(x, y) ((x) + (y))
// x와 y 둘 다 fp일 때 빼기
#define FP_SUB_FP(x, y) ((x) - (y))
// x와 y 둘 다 fp일 때 곱하기
#define FP_MUL_FP(x, y) (((int64_t)(x)) * (y) / F)
// x와 y 둘 다 fp일 때 나누기
#define FP_DIV_FP(x, y) (((int64_t)(x)) * F / (y))
// x는 fp, n은 int일 때 더하기
#define FP_ADD_INT(x, n) ((x) + F * (n))
// x는 fp, n은 int일 때 빼기
#define FP_SUB_INT(x, n) ((x) - F * (n))
// x는 fp, n은 int일 때 곱하기
#define FP_MUL_INT(x, n) ((x) * (n))
// x는 fp, n은 int일 때 나누기
#define FP_DIV_INT(x, n) ((x) / (n))

#endif
```

Pintos 커널은 부동소수점 연산을 지원하지 않기에 이를 위한 매크로들을 정의한 헤더 파일을 만들었다. F는 정수 1을 14번 shift하여 고정소수점으로 만든 것이다. INT_TO_FP는 정수에 F를 곱해 정수를 고정소수점으로 변환하는 것이다. FP_TO_INT_ZERO와 FP_TO_INT_NEAREST는 둘 다 정수로 바꿔주는 연산이지만 0으로 버림인지, 반올림인지에 따라 구현방식이 다르다. FP_ADD_FP 및 FP_SUB_FP는 그냥 소수점끼리 단순히 더하기, 빼기 연산을 하는 것이다. FP_MUL_FP의 경우 곱하는 것인데, $x = a * F$ 이고, $y = b * F$ 이므로 곱하면 $a * b * F^2$ 이 나오므로 원하는 것은 $(a * b) * F$ 이기에 F로 나눠준다. FP_DIV_FP는 반대로 F가 사라지므로 곱해준다. FP_ADD_INT 및 FP_SUB_INT는 정수인 n을 고정소수

점을 바꿔줘야 하기 때문에 F를 곱한다음에 더한다. FP_MUL_INT 및 FP_DIV_INT는 그냥 곱하고 나누면 scale이 유지되기 때문에 그대로 곱하고 나누면 된다.

자료구조 및 전역 변수

```
struct tlist_item {
    int nice;
    int recent_cpu;
    static int load_avg;
};
```

Nice 및 recent_cpu를 thread 구조체 내부에 선언했고, 시스템 부하 평균을 나타내는 load_avg를 thread.c에서 전역변수로 선언했다.

Calculate_priority

```
/* MLFQS 관련 함수들 */
void calculate_priority(struct thread* t) {
    if (t == idle_thread) return;
    // 공식: priority= PRI_MAX - (recent_cpu / 4) - (nice * 2)
    int recent_cpu_div_4 = FP_DIV_INT(t->recent_cpu, 4);
    int nice_mul_2 = t->nice * 2;

    int new_priority = INT_TO_FP(PRI_MAX);
    new_priority = FP_SUB_FP(new_priority, recent_cpu_div_4);
    new_priority = FP_SUB_INT(new_priority, nice_mul_2);

    int new_priority_int = FP_TO_INT_NEAREST(new_priority);

    if (new_priority_int > PRI_MAX) new_priority = PRI_MAX;
    if (new_priority_int < PRI_MIN) new_priority = PRI_MIN;
    t->priority = new_priority_int;
}
```

매 4tick마다 호출되는 recalculate_all_priority() 및 thread_set_nice()에서 호출된다. Idle thread는 계산에서 제외하고 $\text{priority} = \text{PRI_MAX} - (\text{recent_cpu} / 4) - (\text{nice} * 2)$ 의 식에 따라 계산한다. 그리고 Fixed_point.h에서 구현한 매크로들을 활용하였다.

Calculate_recent_cpu()

```
void calculate_recent_cpu(struct thread* t) {
    if (t == idle_thread) return;
    // 공식: recent_cpu = (2*load_avg)/(2*load_avg + 1) * recent_cpu + nice
    int mul2_load_avg = FP_MUL_INT(load_avg, 2);
    int mul2_load_avg_plus1 = FP_ADD_INT(mul2_load_avg, 1);
    int temp = FP_DIV_FP(mul2_load_avg, mul2_load_avg_plus1);
    temp = FP_MUL_FP(temp, t->recent_cpu);
    t->recent_cpu = FP_ADD_INT(temp, t->nice);
}
```

매초마다 recalculate_all_recent_cpu에서 호출되며 최근 CPU 사용량을 계산하는 함수이다. 마찬가지로 $\text{recent_cpu} = (2 * \text{load_avg}) / (2 * \text{load_avg} + 1) * \text{recent_cpu} + \text{nice}$ 공식을 사용하여 thread의 recent_cpu 필드 값을 업데이트 해준다.

Calculate_load_avg()

```
void calculate_load_avg(void) {
    // 공식: load_avg = (59/60)*load_avg + (1/60)*ready_threads
    // Estimate the average of number of thread in READY state.
    // Initialized to 0 when it is booted.
    // load_avg value is updated in every second.
    int ready_threads = list_size(&ready_list);

    if (thread_current() != idle_thread) {
        ready_threads += 1;
    }

    int temp01 = FP_DIV_INT(INT_TO_FP(59), 60);
    int temp02 = FP_DIV_INT(INT_TO_FP(1), 60);

    load_avg = FP_ADD_FP(FP_MUL_FP(temp01, load_avg), FP_MUL_INT(temp02, ready_threads));
}
```

Load_avg를 계산하는 함수로 가장 중요한 것은 ready_list의 크기에 현재 실행 중인 thread도 Ready 상태로 간주해서 추가하여 계산한다는 것이다. 매초마다 호출된다.

Increment_recent_cpu

```
void increment_recent_cpu(void) {
    struct thread* cur = thread_current();
    if (cur == idle_thread) return;
    cur->recent_cpu = FP_ADD_INT(cur->recent_cpu, 1);
}
```

매 tick마다 thread_tick에서 호출되는 함수로 현재 실행 중인 thread의 recent_cpu를 1 증가 시킨다. Idle thread는 제외한다.

Recalculate_all_priority

```
void recalculate_all_priority(void) {
    for (struct list_elem* e = list_begin(&all_list); e != list_end(&all_list);
         e = list_next(e)) {
        struct thread* t = list_entry(e, struct thread, allelem);
        calculate_priority(t);
    }

    if (list_empty(&ready_list)) return;
    struct thread* highest_priority_thread =
        list_entry(list_front(&ready_list), struct thread, elem);
    int highest_priority = highest_priority_thread->priority;
    struct thread* cur = thread_current();
    if (cur->priority < highest_priority) {
        intr_yield_on_return();
    }
}
```

All_list의 모든 thread를 순회하며 각 thread마다 calculate_priority를 호출한다. 그리고 가장 높은 우선순위의 thread와 현재 thread를 비교하여, 더 우선순위가 높다면 intr_yield_on_return을 호출한다. 매 4tick마다 호출된다.

Recalculate_all_recent_cpu

```
void recalculate_all_recent_cpu(void) {
    for (struct list_elem* e = list_begin(&all_list); e != list_end(&all_list);
         e = list_next(e)) {
        struct thread* t = list_entry(e, struct thread, allelem);
        calculate_recent_cpu(t);
    }
}
```

매 초마다 호출되고 All_list를 순회하며 각 thread마다 calculate_recent_cpu를 호출한다.

Thread_set_nice

```
void
thread_set_nice (int nice UNUSED)
{
    thread_current()->nice = nice;
    calculate_priority(thread_current());
    thread_yield();
}
```

Nice값을 변경하는 함수로 우선순위를 재계산하여 더 높은 우선순위의 스레드에게 양보할 수 있도록 한다.

Thread_get_nice

```
int
thread_get_nice (void)
{
    return thread_current()->nice;
}
```

현재 thread의 Nice값을 반환한다. 추가적으로 수정하지 않았다.

Thread_get_load_avg

```
int
thread_get_load_avg (void)
{
    int temp = FP_MUL_INT(load_avg, 100);
    return FP_TO_INT_NEAREST(temp);
}
```

추가적으로 수정하지 않았다.

Thread_get_recent_cpu

```
int
thread_get_recent_cpu (void)
{
    struct thread* cur = thread_current();
    int temp = FP_MUL_INT(cur->recent_cpu, 100);
    return FP_TO_INT_NEAREST(temp);
}
```

Cpu 사용량을 반환하는 함수로 추가적으로 수정하지 않았다.

Thread_tick 함수 내부

```
#ifndef USERPROG
    if (thread_prior_aging == true) {
        thread_aging();
    }

    if (thread_mlfqs == true) {
        int64_t cur_ticks = timer_ticks();
        increment_recent_cpu();

        if (cur_ticks % TIMER_FREQ == 0) { // 매 초
            calculate_load_avg();
            recalculate_all_recent_cpu();
        }
        if (cur_ticks % 4 == 0) {
            recalculate_all_priority();
        }
    }
#endif
}
```

매 틱마다 현재 thread의 CPU 사용량을 1씩 늘리고, 매초마다 시스템 부하를 재계산하는 calculate_load_avg, 모든 thread의 CPU 사용량을 재계산하는 recalculate_all_recent_cpu를 호출한다. 4tick마다 모든 thread의 우선순위를 재계산하는 recalculate_all_priority를 호출한다.

Init_thread

```
initial_thread->recent_cpu = INT_TO_FP(0);
initial_thread->nice = 0;
load_avg = INT_TO_FP(0);
```

가장 처음의 Thread 경우 recent_cpu, nice, load_avg 모두 0으로 초기화시킨다.

Thread_create

```
if (thread_mlfqs) {
    t->nice = thread_current()->nice;
    t->recent_cpu = thread_current()->recent_cpu;
    calculate_priority(t);
}
```

새 thread의 경우 부모 thread의 nice, recent_cpu 값을 상속시키고 초기 우선순위를 계산한다.

C. 시험 및 평가 내용

- priority-lifo.c 코드 및 priority-lifo 테스트 결과 분석

```
thread_set_priority (PRI_DEFAULT + THREAD_CNT + 1);
for (i = 0; i < THREAD_CNT; i++)
{
    char name[16];
    struct simple_thread_data *d = data + i;
    snprintf (name, sizeof name, "%d", i);
    d->id = i;
    d->iterations = 0;
    d->lock = &lock;
    d->op = &op;
    thread_create (name, PRI_DEFAULT + 1 + i, simple_thread_func, d);
}

thread_set_priority (PRI_DEFAULT);
/* All the other threads now run to termination here. */
```

코드를 분석하자면 가자 메인 스레드가 우선순위를 높여 thread를 생성한다. Thread 0이 32, thread 15가 47 이런 순으로 계속 높아지는 것이다. 아무것도 실행이 안되다가 그러나 메인 스레드가 PRI_DEFAULT로 우선순위가 낮춰진 이후, 우선순위가 높은 thread부터 선택하기 때문에 thread 15가 실행된다. 이후 계속 우선순위대로 먼저 실행되고 종료되는 모습을 확인할 수 있다.

```

Executing 'priority-lifo':
(priority-lifo) begin
(priority-lifo) 16 threads will iterate 16 times in the same order each time.
(priority-lifo) If the order varies then there is a bug.
(priority-lifo) iteration: 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15
(priority-lifo) iteration: 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14
(priority-lifo) iteration: 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13
(priority-lifo) iteration: 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12
(priority-lifo) iteration: 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11
(priority-lifo) iteration: 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
(priority-lifo) iteration: 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9
(priority-lifo) iteration: 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
(priority-lifo) iteration: 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
(priority-lifo) iteration: 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
(priority-lifo) iteration: 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
(priority-lifo) iteration: 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
(priority-lifo) iteration: 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
(priority-lifo) iteration: 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
(priority-lifo) iteration: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
(priority-lifo) iteration: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
(priority-lifo) end
Execution of 'priority-lifo' complete.
Timer: 31 ticks
Thread: 0 idle ticks, 31 kernel ticks, 0 user ticks
Console: 1557 characters output
Keyboard: 0 keys pressed
Powering off...

```

LIFO라는 이름에서 알 수 있듯이 나중에 생성된 thread가 높은 우선순위를 가지고 먼저 실행되고 종료되는 결과를 볼 수 있다.

- make check 수행 결과를 캡처하여 첨부

```

pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-change-2
FAIL tests/threads/priority-donate-one
FAIL tests/threads/priority-donate-multiple
FAIL tests/threads/priority-donate-multiple2
FAIL tests/threads/priority-donate-nest
FAIL tests/threads/priority-donate-sema
FAIL tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-aging
pass tests/threads/priority-condvar
FAIL tests/threads/priority-donate-chain
pass tests/threads/mlfqs-load-1
pass tests/threads/mlfqs-load-60
pass tests/threads/mlfqs-load-avg
pass tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
pass tests/threads/mlfqs-nice-2
pass tests/threads/mlfqs-nice-10
pass tests/threads/mlfqs-block
7 of 29 tests failed.

```

Donate 관련 7개 test 빼고 모두 수행되는 모습을 확인할 수 있다.